Lecture 21

Filtering SAM files

SAM Flags

Selecting alignments by some characteristics is a very common requirement. At some point you will need:

1. Alignments that align to forward strand.
2. Alignments that overlap witha coordinate.
3. Alignments where the mate does not align.
4. Alignments that came from the second file.
5. *... and various combinations thereof.*

Remote access

Get a demonstration BAM file.

```
wget http://data.biostarhandbook.com/bam/demo.bam

samtools index demo.bam

samtools view demo.bam | head
```

adding the `-c` flag makes `samtools view` report counts rather than the entire alignment.

```
samtools view -c demo.bam
31571
```

Filtering flags: keep (-f) and remove (-F)

A suprisingly awkward explanation:

```
samtools view
```

provides the following help:

```
-f INT only include reads with all  of the FLAGs in INT pr
-F INT only include reads with none of the FLAGS in INT pr
```

Better defined as:

`-f` selects alignments if **ALL** flags match
`-F` removes alignments if **ANY** the flags match

Using filtering flags

```
# What is the unmapped flag?
samtools flags UNMAP
0x4     4       UNMAP

# How many total entries.
samtools view -c demo.bam
31571

# How many entries that are not mapped.
samtools view -c -f 4 demo.bam
226

# How many not-unmapped alignments.
samtools view -c -F 4 demo.bam
31345
```

Annoying circuitousness of the SAM format

“ *Premature optimization - the root of all evil.* ”

You see there is no flag for **aligned read**.

There is only a flag for **unmapped read**.

Founding fathers force us to use: **unmapped**

You want aligned reads? Just say: give me reads that are **NOT UNMAPPED**

Easy peasy 😱

Silver lining

Few people understand the SAM format.

So once you do, you become special 😎 and valuable to society

Filtering with flags

```
samtools flags

Flags:
    0x1   PAIRED         .. paired-end (or multiple-segment
    0x2   PROPER_PAIR    .. each segment properly aligned a
    0x4   UNMAP          .. segment unmapped
    0x8   MUNMAP         .. next segment in the template un
    0x10  REVERSE        .. SEQ is reverse complemented
    0x20  MREVERSE       .. SEQ of the next segment in the
    0x40  READ1          .. the first segment in the templa
    0x80  READ2          .. the last segment in the templat
    0x100 SECONDARY      .. secondary alignment
    0x200 QCFAIL         .. not passing quality controls
    0x400 DUP            .. PCR or optical duplicate
    0x800 SUPPLEMENTARY  .. supplementary alignment
```

What the heck is a proper pair?

According to the SAM spec flag = 2 means

> “ each segment properly aligned according to the aligner ”

But what does this actually mean?

Feels painfully subjective: *"according to the aligner..."*

Turns out nobody knows what this actually means. Ahem...

On the second flag our prophet, Heng Li, separated alignments into proper and improper, and he said:

According to me

this alignment is proper

And it was so. And we saw that it was good.

What does proper pair usually mean?

1. Both reads in pair align

2. Read pairs in the "illumina" orientation

```
------>                 <-------
```

3. The template lenght of the read pair is within 4 standard deviations of the last block of read pairs.

I suggest to not use this flag since it is not defined.

Use flag combinations!

One flag alone usually does not do what you think it does ⚡! Accept that and life will be easier.

To get reads aligned on the **forward** strand: remove unaligned with `-F` `UNMAP` then remove reverse stand with `-F` `REVERSE`:

```
samtools view -c -F 4 -F 16 demo.bam
18055
```

The two flags can be combined into one:

```
samtools view -c -F 20 demo.bam
18055
```

Select into BAM file

Make another BAM file with `-b` with reads that align on forward strand:

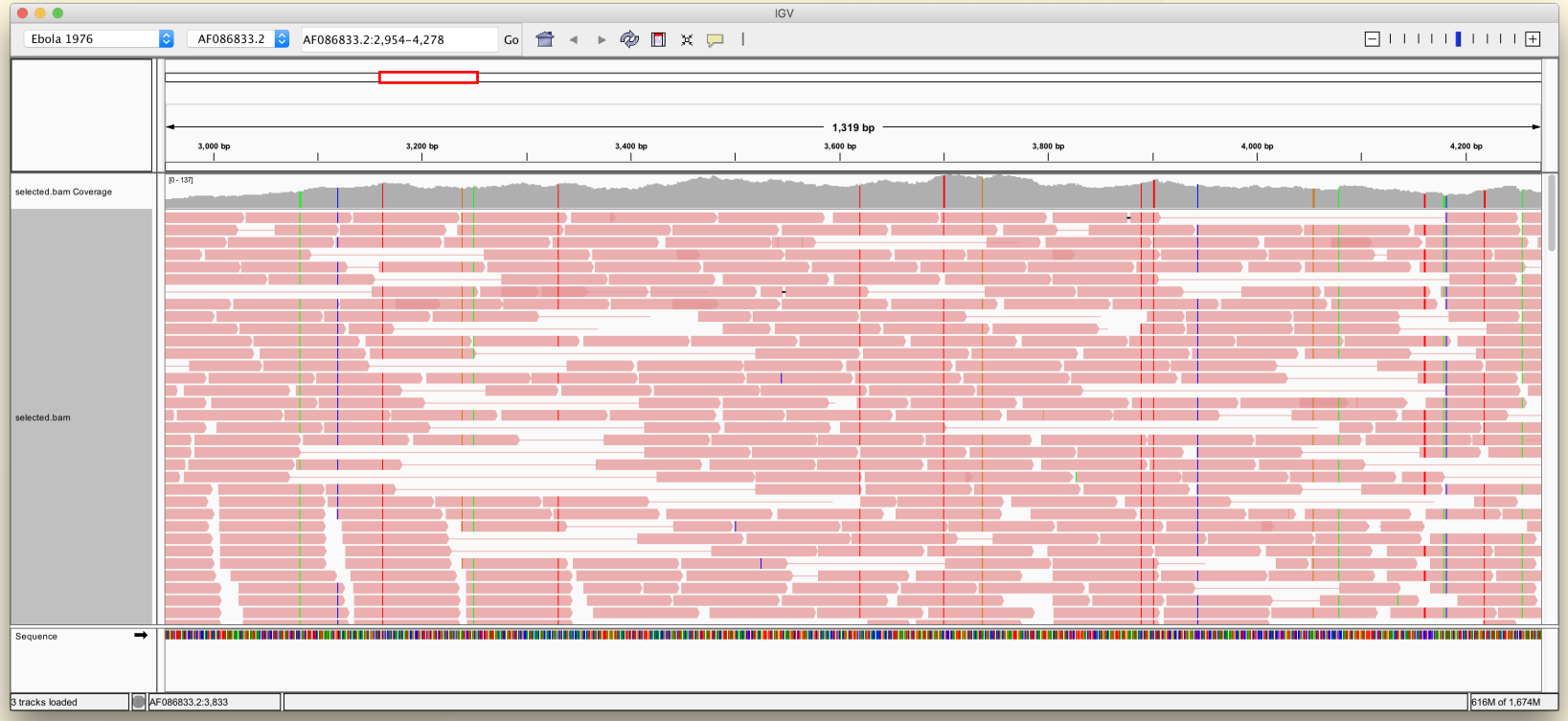
```
samtools view -b -F 4 -F 16 demo.bam > selected.bam
samtools index selected.bam
```

Selecting alignments where read aligns on the forward strand.

Does it do what you think it does?

View the results as pairs in IGV.

Is this what you wanted?



There are pairs where both reads align on the forward strand. How is that possible?

Fact or fiction?

One of the hardest things to sort out when unexpected alignments show up.

Is this **real** or some weird **artifact** of the experiment? Sometimes is the former, many times is the latter.

We don't really understand how DNA works. We don't really understand how life works.

Even this data, published three years ago is filled to the brim with unexplored information.

Valid fragments on both strands

Align not on reverse with the mate on the reverse:

```
samtools view -b -F 4 -F 16 -f 32 demo.bam  > selected.bam
samtools index selected.bam
```

To invert the meaning swap `-f` with `-F` where applicable.

Aligned on reverse with mate on the forward:

```
samtools view -b -F 4 -f 16 -F 32 demo.bam  > selected.bam
samtools index selected.bam
```

Reverted selection



Learning from other sciences

Physics values symmetry - it states that assymetrical formulas indicate that the concepts are not well defined.

In the collapsed representation the first selection is:

```
samtools view -F 20 -f 32 demo.bam
```

The inverted selection becomes:

```
samtools view -f 16 -F 36 demo.bam
```

The first representation is the exact opposite of the second. But there is no symmetry 🤔

Secondary alignment

A read may have multiple mappings

One alignment may be designated as primary the others as secondary alignments.

There is a surprising ambiguity and uncertainty as to what gets reported for each aligner.

Some aligners will not report this, others do.

Supplementary alignment

A better word: chimeric alignment.

Represents a split alignment where a read aligns in two different locations with no overlap.

Turned to have high practical utility. Lots of reasons for sequences to fuse naturally or as a protocol.

Not all aligners can find these types of alignments!

`bwa mem` can. All the more reason we like it.

Supplementary (chimeric) alignments.

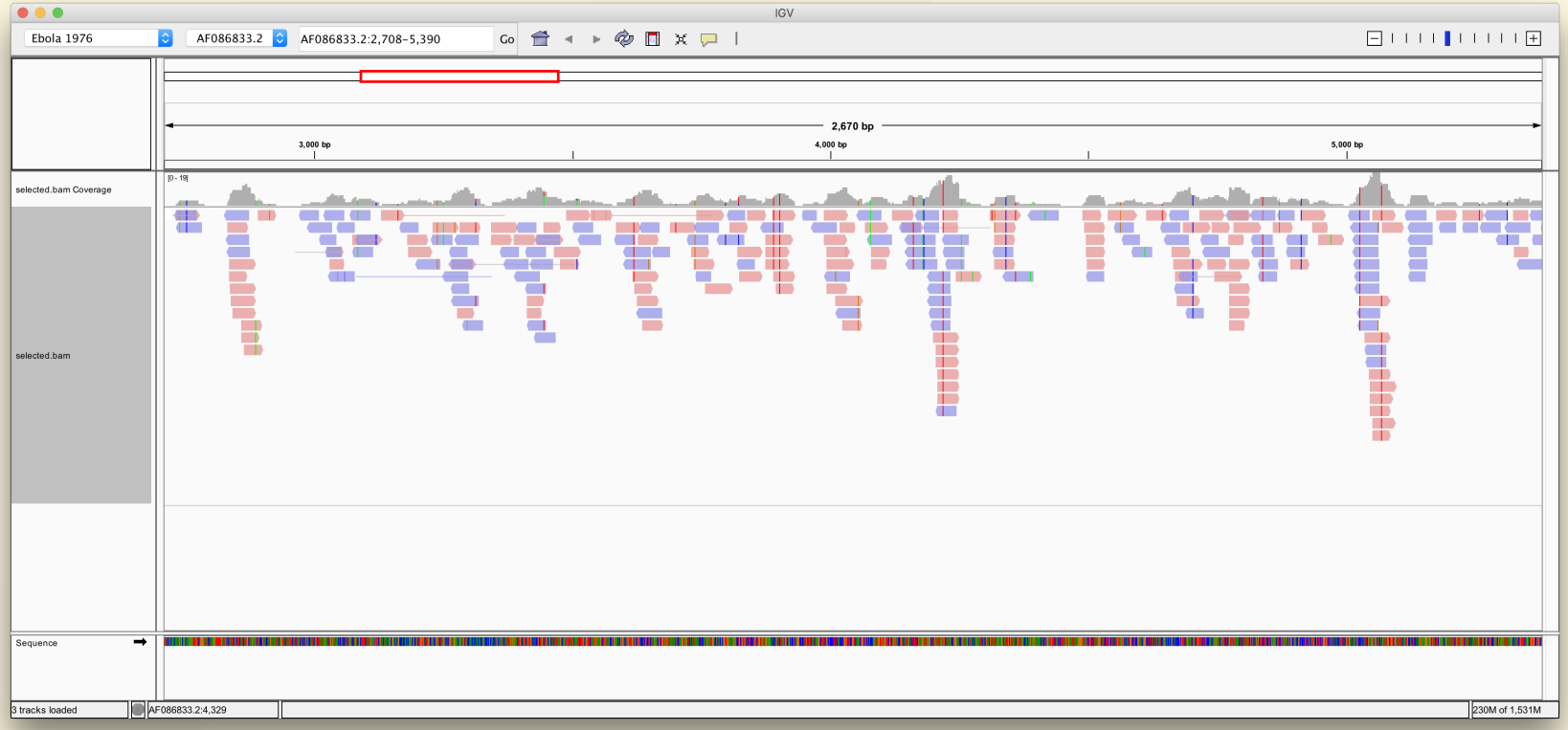
```
READ 10bp
---------->
```

A **chimeric aligment** aligns the read as if it were cut in two and both segments align in different locations.

```
=======================================
      5                       5
    ----->                  ----->
      3                       7
     --->                  ------->
      8                       2
  -------->                  -->
```

Investigate chimeric alignments

```
samtools view -b -F 4 -f 2048 demo.bam > selected.bam
samtools index selected.bam
```



Soft and hard clips

Indicated by `S` or `H` in CIGAR strings:

```
10S100M
10H100M
```

Will appear at the start or end.

Indicate that the first `N` bases of the read did not align. There is low penalty (could be even zero).

Very useful feature. Not all aligners can clip!

`bwa mem` can.

Difference between soft and hard clip

Soft clipped bases are included in the SAM file.

Hard clipped bases are not included the SAM file.

If the read is `ATGC` then the CIGAR

- `1S3M` will have `ATGC` in the sequence column.
- `1H3M` will have `TGC` in the sequence column.

Read groups

Tagging each alignment with sample information.

- `ID`: unique id
- `SM`: sample name
- `LB`: library name

You can tag during alignment:

```
TAG='@RG\tID:xyz\tSM:Ebola\tLB:patient_100'
```

Add the tags during alignment

```
bwa mem -R $TAG $REF $R1 $R2 | samtools sort > bwa.bam
```

Add readgroups later

```
TAG='@RG\tID:abc\tSM:Ebola\tLB:patient_101'

samtools addreplacerg -m overwrite_all -r $TAG demo.bam -O
```

See the headers:

```
samtools view -H new.bam
```

prints:

```
@HD     VN:1.3  SO:coordinate
@SQ     SN:AF086833.2   LN:18959
@PG     ID:bwa  PN:bwa  VN:0.7.16a-r1181        CL:bwa mem
@RG     ID:abc  SM:Ebola        LB:patient_101
```

Each alignment is now tagged

```
samtools view new.bam | cut -f 17 | head -1
```

The format keeps track of the origins of each read:

```
RG:Z:abc
```

The unique id `abc` identifies sample name `Ebola` and library `patient_101` from the header.

Many tools rely on the presence of read groups.